Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente–servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guia practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación

- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos) Este módulo se desarrolla de forma independiente Actividad

1. Crear un archivo:
   servidor.py
2. Crear un archivo auxiliar:
   entrada_servidor.txt
3. Implementar un programa que:
   - Revise periódicamente el archivo entrada_servidor.txt.
   - Detecte si existe contenido nuevo.
   - Procese el mensaje:
     - Ejemplo: convertir texto a mayúsculas o invertirlo.
   - Escriba el resultado en:
   salida_servidor.txt
4. Condiciones:
   - Debe ejecutarse continuamente (simulación de servicio activo).
   - No debe reprocesar el mismo mensaje.
   - Debe manejar archivos vacíos.

◆ **Parte 2: Desarrollo del CLIENTE (1 hora)**

Se desarrolla sin necesidad de ver el código del servidor Actividad

1. Crear archivo:
   cliente.py
2. Implementar un programa que:
   - Solicite un mensaje al usuario.
   - Escriba el mensaje en:
   entrada_servidor.txt
   - Espere un tiempo (simulando latencia).
   - Lea la respuesta desde:
   salida_servidor.txt
   - Muestre el resultado en pantalla.
3. Condiciones:
   - No debe asumir cómo funciona el servidor internamente.
   - Solo debe conocer los archivos de entrada y salida.

◆ **Parte 3: Integración del sistema (25 minutos)**

1. Ejecutar primero el servidor.

2. Ejecutar el cliente.
3. Enviar varios mensajes.
4. Verificar:
    - Flujo correcto
    - Respuesta adecuada

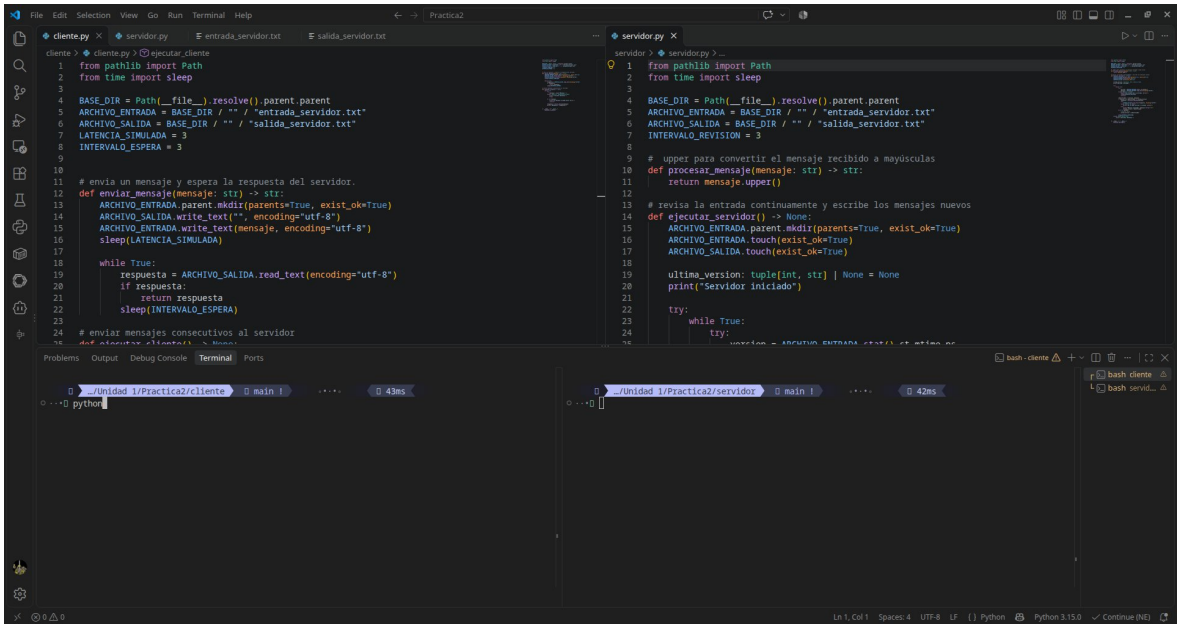
NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

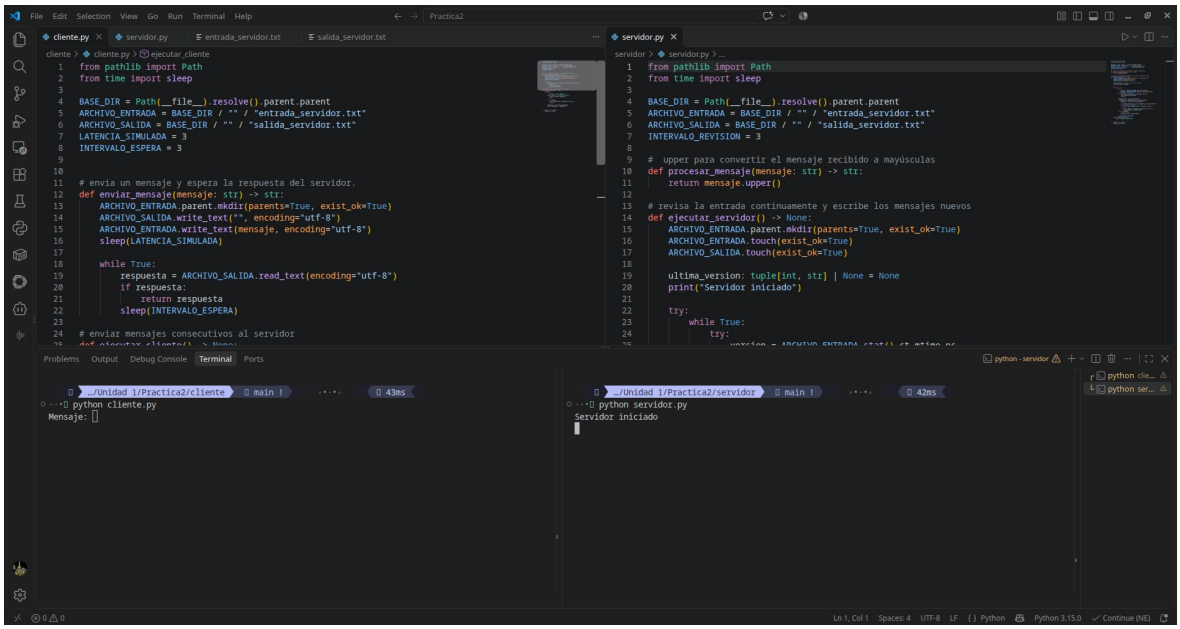
6. Resultados esperados:

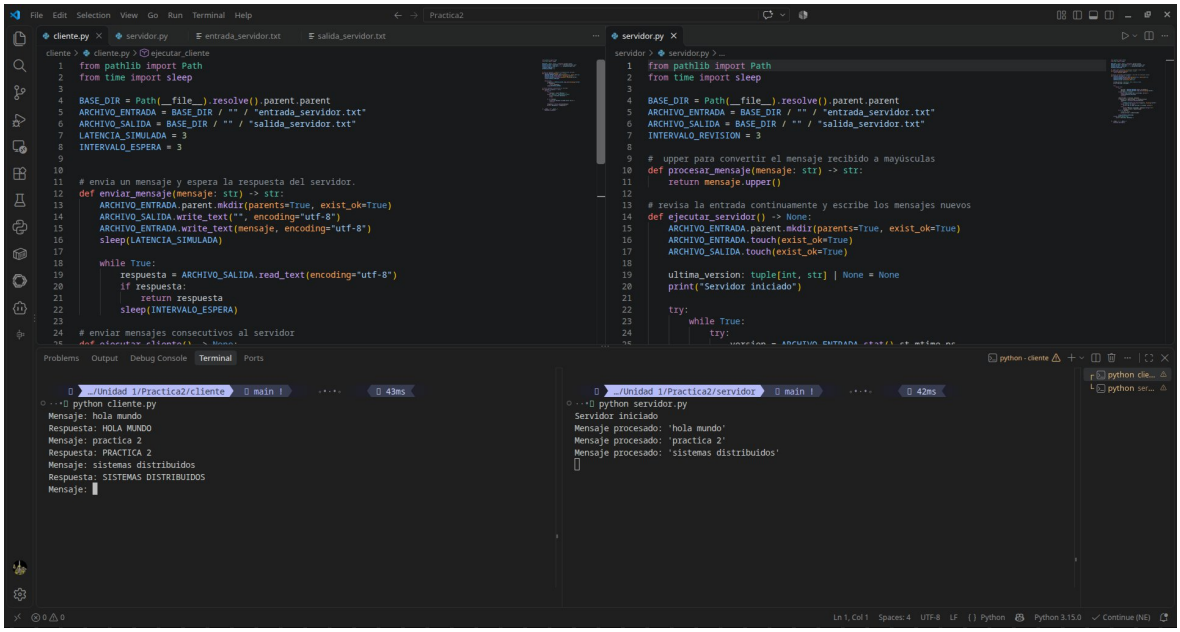
Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git).
  Enlace al código fuente

- Capturas de pantalla que evidencien: (a) el archivo entrada_servidor.txt siendo llenado por el cliente, (b) salida_servidor.txt con el resultado procesado, (c) al menos 3 mensajes procesados correctamente.







- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).

7. Preguntas de Control:

¿Qué significa que cliente y servidor estén desacoplados?

Que cada uno funciona de forma independiente, sin conocer el código ni la implementación interna del otro. Solo comparten un contrato: los nombres y el formato de los archivos.

¿Qué ventajas tiene desarrollar componentes por separado?

Permite trabajo en paralelo por distintos equipos, facilita las pruebas y el mantenimiento, y se puede modificar o reemplazar un componente (otro lenguaje, otra lógica) sin afectar al otro.

¿Qué problemas pueden surgir al integrar sistemas independientes?

Formatos de mensaje incompatibles, diferencias en codificación de texto (UTF-8), rutas de archivos distintas, desajustes de tiempos de espera y errores difíciles de rastrear porque cada parte asume cosas diferentes.

¿Qué sucede si el servidor no está ejecutándose?

El mensaje queda escrito en entrada_servidor.txt, pero nadie lo procesa. El cliente no recibe respuesta (o lee una antigua) y debería terminar por tiempo de espera agotado.

¿Cómo se simula la comunicación en esta práctica?

Mediante comunicación indirecta por medio compartido: los archivos de texto hacen de canal. El servidor hace polling (revisa periódicamente), y el cliente simula latencia con una espera antes de leer la respuesta.

¿Qué limitaciones tiene este modelo frente a uno real?

No es tiempo real (depende del intervalo de revisión), hay riesgo de conflictos de escritura y lectura simultánea, no hay confirmación de entrega ni manejo de errores, el polling consume recursos, y depende de un disco o carpeta compartida (no escala por red ni es seguro).

¿Cómo se podría mejorar la sincronización?

Usando bloqueo de archivos (file locks), identificadores únicos por mensaje, archivos de confirmación (ACK) o banderas de estado, o reemplazando los archivos por sockets TCP/HTTP, colas de mensajes (RabbitMQ, Kafka) o notificaciones de cambios del sistema de archivos.

¿Qué pasaría si múltiples clientes usan el sistema?

Habría condiciones de carrera: los clientes sobrescribirían los mensajes de otros, las respuestas se mezclarían y un cliente podría leer la respuesta de otro. Se resolvería con archivos separados por cliente o ID de sesión, o con una cola de mensajes.

8. Evaluación

NO APLICA

9. Bibliografía

Semana 2. Guaman, Jose. 2026

10. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC